Running HybridQC

A practical guide to the normalized JSON interface in HybridQC. Python prepares inputs and reads results. The compiled Rust engine advances simulated time and enforces dependencies and resource capacity.

Access. These instructions require a simulator source checkout. Source release is pending institutional authorization, as stated in the paper. The supporting data and result-reading examples can be used independently. The accompanying examples are small teaching cases with explicit assumed service times.

1. Prepare the environment

Use Python 3.10 or newer, Rust/Cargo, and a C/C++ build toolchain. Start in the simulator checkout. A virtual environment keeps the package and its compiled extension together.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip maturin
python -m pip install -r requirements.txt
python -m pip install -e .
CARGO_BUILD_JOBS=4 CARGO_INCREMENTAL=0 \
CARGO_PROFILE_RELEASE_DEBUG=0 \
python -m maturin develop --release --locked \
  --features python-module \
  --manifest-path \
  extensions/hcu_native_rust/crates/hcu_native_py/Cargo.toml
```

The build-worker setting limits compiler processes. It does not set the number of simulated devices or the parallelism of an experiment. This guide uses the compiled engine for every simulation.

2. Check the command and run one job

Place the accompanying **examples/** directory at the simulator checkout root, or replace the paths below with its actual location. The input files are ready to run.

```
python -m hybridqc.cli.hcu_sim --help
python -m hybridqc.cli.hcu_sim \
  --config examples/baseline-config.json \
  --jobs examples/one-job.json \
  --execution microstep \
  --output runs/tutorial-one.json
```

The command creates the output parent directory and writes a JSON summary. This example completes one job at simulated time 9.0 s. It performs no hardware submission and needs no cloud credentials. The input seed is 7, and quantum error correction is disabled.

3. Understand the two input files

config.json describes resources and execution settings. **jobs.json** is an array of jobs. In these examples each job is a sequence of stages. Several jobs compete for the same resources.

Configuration field	Meaning
schema_version	The normalized input-schema version. The examples use 1.
graph.nodes	Named resources, their class (base), device count, memory capacity, and qubit capacity.
graph.edges	Directed links with src, dst, latency_s, and bandwidth_Bps.
sim.horizon_s / seed	Simulated-time horizon and random seed. A horizon is not a wall-clock timeout.
sim.dqc_allocation_mode	DQC allocation setting. The examples use monolithic_cohort and consume the full 32-qubit device per DQC stage.
policy.name / params	Scheduling policy and its parameters. The examples use fifo.
runtime	Execution strategy and optional model features. The examples set microstep and disable QEC.

A stage in jobs.json

```
{
  "stage": "DQC",
  "device": "DQC",
  "duration_s": 2.0,
  "qubits": 32
}
```

A job provides job_id, arrive_time_s, workload, and tasks. Each task names a stage, its target resource, and its service time. Memory demand uses mem_gb, qubit demand uses qubits, and payload size uses payload_bytes. Seconds, bytes, and bytes/s are distinct units. Device counts describe the simulated machine, not the CPU cores running the simulator.

For a hardware-grounded study, workload construction and the selected backend profile determine stage service times before execution. Scheduler waiting is then produced by contention. Keep the profile identity with the input and avoid adding measured provider turnaround to a physical-occupancy term.

The teaching inputs have no transfer stages and an empty edge list. They isolate stage ordering and capacity. Modeling communication requires declared links and transfer tasks. The command above consumes the normalized JSON files supplied here.

4. Read, compare, and retain results

Output field	How to use it
jobs_submitted / jobs_completed	Check completion before interpreting runtime or speedup.
job_end_s	Per-job finish times keyed by job ID. Subtract each job's arrival to get its completion latency.
total_time_s	Elapsed simulated time reported by the engine. For these complete, zero-arrival examples, it equals the final job finish.
makespan_mean_s / makespan_p95_s	Mean and reported p95 of completed-job latencies. These differ from total run duration.
stage_count / stage_service_time_s	Completed work and service summed by stage type.
stage_wait_time_s	Waiting summed over stage instances. Overlapping waits can exceed elapsed run time.
reservation_trace	Stage enqueue/start/finish times, resource identity, and exact timing fields for a small diagnostic run.

```
python - <<'PY'
import json
r = json.load(open("runs/tutorial-one.json"))
print(r["jobs_completed"], r["total_time_s"])
print(r["stage_service_time_s"])
print(r["stage_wait_time_s"])
PY
```

Matched comparisons. Keep jobs, arrivals, seed, service profiles, and horizon fixed. Change one resource or policy setting, write to a new path, and compare completion counts first. The examples guide provides a four-configuration comparison and checked outputs.

Common problems

Package not found: activate the environment where the editable package was installed. **Native extension missing:** build the Rust extension in that same environment. **Input rejected:** use an object for the configuration, an array for jobs, and resource names that match the graph. The CLI prints the error and exits with status 2.

Incomplete run: check submitted versus completed jobs, the horizon, resource requirements, and links before changing a setting. A returned summary alone does not establish completion. **Unexpected speedup:** inspect service and waiting separately and confirm both cases completed the same work.

Save the evidence needed for the comparison

Retain exact inputs, seed, backend-profile identity, source/build identity, completion status, and the summary. These small examples produce full stage traces. For large studies, use the campaign's compact retention path. Execution strategy and retention are separate settings.